DEEP REINFORCEMENT LEARNING PROJECT - SNAKE

Federico Saporiti, Control Systems Engineer

April 20, 2026

Abstract

This report details the development of Reinforcement Learning agents to play the game of Snake in fully and partially observable environments. The primary solution implements a Double Deep Q-Network (DQN) utilizing an ϵ-greedy exploration strategy to approximate Q-values for navigational decision-making. The performance of the DQN is then benchmarked against a custom deterministic heuristic baseline and an alternative Advantage Actor-Critic (A2C) architecture. Ultimately, this comparative approach evaluates the efficacy of these specific learning algorithms under strict observability constraints.

1. Heuristic Baseline

To establish a performance benchmark for reinforcement learning algorithms, a deterministic heuristic policy was implemented. This baseline uses a naive greedy algorithm that calculates only the immediate coordinate displacement between the snake's head and the target fruit. The decision logic first prioritizes horizontal alignment, moving along the X-axis to reduce the distance. Then, it aligns the Y-axis once the X-coordinates match.

This heuristic operates without spatial awareness or advanced pathfinding mechanics. It ignores walls and does not attempt to avoid colliding with its own body segments.

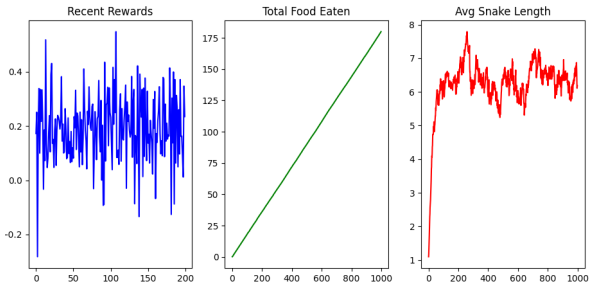

Figure 1. Performance metrics of the naive greedy baseline over 1000 iterations

Figure 1 shows the evaluation of this baseline over 1,000 iterations and clearly illustrates the inherent flaws of a purely greedy strategy. The total amount of food eaten increases linearly, reaching approximately 180 units by the end of the run. However, the average snake length metric reveals a severe performance ceiling. After an initial phase of rapid growth in the first 150 iterations, the average length stabilizes, fluctuating between approximately 5 and 7 segments for the rest of the evaluation.

These metrics demonstrate that, once the snake reaches 6 to 8 segments, its lack of self-avoidance logic causes it to trap and collide with itself consistently, forcing a reset, preventing long-term survival or substantial growth.

2. Environment Modifications

To optimize the learning process, the static reward structure was updated to introduce dynamic step and food rewards that scale with the snake's current size. The step reward, which acts as a penalty, is dynamically adjusted by dividing it by $0.1 \times$ length. This mechanism penalizes the agent for taking unnecessarily long paths when it is short, while affording it the necessary flexibility to take longer, safer routes to avoid self-collision as its body grows. To ensure the agent remains motivated to increase its size as it gets bigger, the food reward is incremented by a factor of $+0.15 \times$ length. This encourages the snake to prioritize eating rather than prematurely terminating the episode when navigation becomes tough.

Furthermore, a substantial optimization was applied to the state extraction mechanism in the partially observable environment. The original `to_state` method, which utilized a computationally expensive `for` loop to extract local grids, was overridden. The custom `partial_to_state` function pads the boundaries and utilizes vectorized array slicing to extract ego-centric views, significantly accelerating the generation of the state tensor across parallel boards.

Finally, an action-masking utility (`get_action_mask`) was implemented to prevent the agent from executing inherently fatal moves. By evaluating the immediate coordinates around the snake's head and applying a severe negative penalty to the Q-values of invalid actions (such as reversing into its own body or moving directly into a wall), the net-

work's exploration is focused exclusively on viable paths, thereby reducing wasted training cycles.

In addition to these changes, the scale of the simulation was increased to provide a more challenging learning environment. The size of the board was increased to a 10×10 area. This significantly increases the state space, providing a more rigorous test of the agents' pathfinding capabilities and requiring it to develop long-term survival strategies in a broader arena.

3. Deep Q-Network and ϵ-Greedy Strategy

The core learning algorithm implemented is a Double Deep Q-Network (DQN) that approximates the optimal action-value function. The architecture relies on a Multi-Layer Perceptron (MLP) consisting of three dense layers with 128, 256, and 64 units, respectively, utilizing ReLU activations. This network takes the processed state tensor as input and returns four discrete Q-values corresponding to the possible directional movements (Up, Right, Down, Left).

Algorithm 1 Double Deep Q-Network: Learning and Bellman Update

Input: Experience batch (S, A, R, S'), penalty tensor $P(S')$, current step t
// Decoupled action selection and evaluation
Predict $Q'_{vals} = Q_{online}(S')$
Select optimal next actions $A' = \arg\max(Q'_{vals} + P(S'), \text{axis} = -1)$
Calculate Bellman target $Y = R + \gamma Q_{target}(S', A')$
Extract predicted Q-values for taken actions A: $Q_{pred} = Q_{online}(S, A)$
Compute Huber Loss $\mathcal{L}$ between targets Y and predictions Q_{pred}
Apply gradients to update θ_{online}
if $t \bmod 100 == 0$ **then**
  Update target network: $\theta_{target} \leftarrow \theta_{online}$
end if

To effectively navigate the trade-off between exploring the environment and exploiting learned knowledge, the agent employs an ϵ-greedy strategy. At the beginning of training, the exploration rate (ϵ) is initialized at 1.0, ensuring that the agent acts completely randomly to gather diverse initial experiences. During each iteration, ϵ is exponentially decayed by a factor of 0.99992. The decay process is bounded by a minimum threshold ($\epsilon_{\min}$) that varies depending on the observability constraints. For the fully observable environment, $\epsilon_{\min}$ is set to 0.001, allowing the agent to almost exclusively exploit its heavily refined global policy in the later stages of training (note that the decay rate was chosen to be 0.99992 because $\log(0.001)/\log(0.99992) = 86,343$ iterations and the agent was trained on a set of 80,000 iterations). Conversely, the partially observable model employs a significantly higher $\epsilon_{\min}$ of 0.1. This higher threshold requires the agent to maintain a 10% exploration rate to compensate for its limited local vision and prevent it from getting stuck in repetitive loops.

A key feature of this implementation is the integration of the deterministic action mask with the ϵ-greedy action selection process.

- **Exploitation Phase:** The mask applies a massive negative penalty to the Q-values of invalid moves (e.g., hitting a wall or the snake's own body). The agent then selects the highest remaining Q-value, ensuring the chosen action is both optimal and immediately safe.
- **Exploration Phase:** Instead of picking a completely random integer, the algorithm generates random noise and applies the same action mask. Taking the maximum value of this masked noise ensures the agent explores randomly, but strictly within the subset of survivable actions.

Finally, to calculate the loss and update the network, the implementation utilizes a Double DQN approach. The online network, denoted as Q_{online}, evaluates the next state s' to select the optimal next action a'. This selection mathematically incorporates the action mask $M(s', a)$ to strictly heavily penalize and prevent invalid moves:

$$a' = \arg\max_a \left(Q_{online}(s', a) + M(s', a)\right)$$

While the online network selects the action, a synchronized target network Q_{target} (which has its weights updated to match the online network every 100 iterations) calculates the Bellman target value Y for that specific action:

$$Y = R + \gamma Q_{target}(s', a')$$

where R represents the immediate reward and $\gamma = 0.975$ is the discount factor. The network's weights are then updated by minimizing the Huber loss between the calculated target Y and the current prediction $Q_{online}(s, a)$. This intentional decoupling of action selection from action evaluation effectively mitigates the overestimation bias inherent to standard DQNs.

4. Deep Q-Network Training Results

The Double DQN agents were evaluated over 80,000 training iterations, revealing a stark contrast in performance based on the observability constraints. The fully observable agent demonstrated robust, stable learning across all recorded metrics. By perceiving the entire 10×10 board, it successfully developed long-term pathfinding strategies,

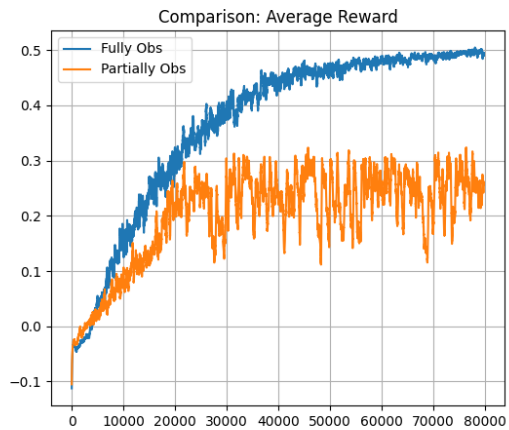

Figure 2. Comparison of Average Reward over 80,000 iterations. The fully observable agent achieves a smooth, high convergence, while the partially observable agent plateaus early with high variance.

reaching an average reward of approximately 0.5 and consistently consuming over 14 fruits per 100 iterations. Consequently, its average length steadily climbed and converged around 18 segments. The smooth plateau observed in the later stages of its training curves is directly attributable to its low minimum exploration rate ($\epsilon_{\min} = 0.001$), which allowed the agent to almost exclusively exploit its highly accurate global policy.

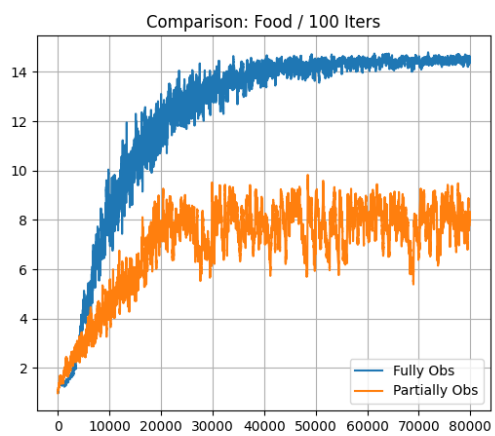

Figure 3. Comparison of Food eaten per 100 iterations.

Conversely, the partially observable agent exhibited a significantly lower performance ceiling and high operational volatility. Constrained by its localized 5×5 ego-centric view, the agent's food consumption is wildly fluctuating. Its average length correspondingly oscillated violently between 13 and 18 segments. This extreme volatility emerges from a

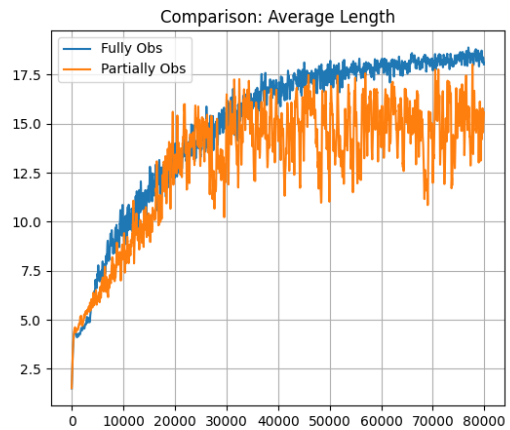

Figure 4. Comparison of Average Snake Length. The partially observable agent struggles to surpass 15 segments consistently due to self-collisions outside its field of view.

specific behavioral loop: once the snake grows long enough that its body clutters its limited visual radius, it adopts a highly conservative "surviving is painful but dying is worse" strategy. Rather than risking exploration into the unknown to find unseen fruit, the agent begins looping tightly upon itself to prioritize immediate survival and avoid fatal collisions. It gets trapped in these safe, repetitive cycles, temporarily halting its food intake. Ultimately, it is the mandatory 10% random exploration noise ($\epsilon_{\min} = 0.1$) that forces the agent to break these safe loops, which frequently results in the snake crashing into its own unseen tail.

To address the high volatility and noisy gradients inherent to the partially observable environment, a different optimization strategy was explored for its neural network.

While the fully observable agent utilized a static learning rate of 5×10^{-4}, the optimizer for the partially observable agent was equipped with an exponential decay schedule.

The learning rate at any given training step t, denoted as η_t, is dynamically calculated using the following discrete staircase function:

$$\eta_t = \eta_0 \times \gamma^{\lfloor \frac{t}{s} \rfloor}$$

Where:

- $\eta_0 = 5.5 \times 10^{-4}$ represents the initial learning rate.
- $\gamma = 0.9$ is the decay rate, representing a 10% reduction per drop.
- $s = 10000$ indicates the decay steps interval.
- $\lfloor \cdot \rfloor$ denotes the floor operation, which mathematically

enforces the step-wise integer division dictated by setting the `staircase=True` parameter.

Because of the floor function, the exponent evaluates to 0 for the first block of steps, meaning the learning rate remains completely constant at 5.5×10^{-4} from iteration 0 to 9,999. Exactly at iteration 10,000, the exponent increments to 1, causing the learning rate to drop instantaneously to 4.95×10^{-4} (which is $5.5 \times 10^{-4} \times 0.9^1$). It maintains this new plateau until iteration 20,000, where it drops again to 4.455×10^{-4}, and so forth.

This dynamic approach was implemented to allow the network to make large, rapid policy adjustments during the initial exploration phase. Crucially, maintaining a higher initial learning rate prevents the optimizer from converging too early into conservative self-looping behaviors when the snake is still short. It forces the short agent to actively explore and learn to pursue food, rather than immediately settling for the local optimum of infinitely looping in an empty space to survive.

Subsequently, the network fine-tunes its weights with progressively smaller updates as training advances. This scheduled, periodic reduction in step size helps stabilize the learning process, preventing the optimizer from drastically overshooting optimal parameter values later on when the extended agent is inevitably trapped in the high-variance survival loops caused by its limited vision.

5. Advantage Actor-Critic (A2C)

To provide another benchmark against the DQN implementation, an Advantage Actor-Critic (A2C) algorithm was implemented. Unlike the value-based DQN, the A2C model utilizes a shared Multi-Layer Perceptron base that branches into two distinct output heads: an *Actor* that outputs policy logits (representing the preference for each of the four actions) and a *Critic* that estimates the scalar value of the current state.

Exploration in this model abandons the ϵ-greedy strategy in favor of stochastic sampling. The deterministic action mask is directly added to the actor's logits, applying a massive negative penalty to fatal moves, after which an action is sampled from the resulting categorical probability distribution. This allows the agent to explore safely based on its learned policy confidence rather than forced random noise.

The optimization of the A2C network relies on a three-part composite loss function. First, the *Advantage* (A) is calculated as the difference between the Bellman target ($R + \gamma V(s')$) and the critic's current value estimate $V(s)$. The total loss $\mathcal{L}$ is computed as:

$$\mathcal{L} = \mathcal{L}_{Actor} + \mathcal{L}_{Critic} - \beta\mathcal{H}$$

Where:

- $\mathcal{L}_{Critic}$ is the Huber loss between the target and $V(s)$, training the critic to accurately predict future returns.
- $\mathcal{L}_{Actor}$ is the policy gradient loss, calculated as $-\log(\pi(a|s)) \times \texttt{stop_gradient}(A)$. The `stop_gradient` operation is crucial; it prevents the actor's optimization step from incorrectly backpropagating through the critic's value predictions, thereby stabilizing training.
- $\mathcal{H}$ is an entropy bonus (weighted by $\beta = 0.01$), calculated from the softmax probabilities of the logits to actively penalize premature certainty and encourage continued exploration of the state space.

Algorithm 2 Advantage Actor-Critic: Learning and Advantage Update

Input: Experience batch (S, A, R, S'), current state value $V(S)$, masked logits L_{masked}
Predict next state values $V(S')$ using network(S')
Calculate Bellman Target $Y = R + \gamma V(S')$
Calculate Advantage $\mathcal{A} = Y - V(S)$
$\mathcal{L}_{Critic} = \text{Huber}(Y, V(S))$
Calculate log probabilities: $\log P = \log(\text{softmax}(L_{masked}))$
Extract log probabilities of taken actions A: $\log P_A$
$\mathcal{L}_{Actor} = -\frac{1}{B}\sum(\log P_A \times \text{stop_gradient}(\mathcal{A}))$
Calculate probabilities: $P_{probs} = \text{softmax}(L_{masked})$
$\mathcal{H} = -\frac{1}{B}\sum(\sum(P_{probs} \times \log P))$
$\mathcal{L}_{Total} = \mathcal{L}_{Actor} + \mathcal{L}_{Critic} - \beta \times \mathcal{H}$
Compute gradients $\nabla_\theta \mathcal{L}_{Total}$ and apply using optimizer

6. A2C Performance Analysis

The Advantage Actor-Critic (A2C) architecture was evaluated over 80,000 training iterations, demonstrating a distinct performance gap dictated by the environment's observability constraints. In the fully observable environment, the A2C agent exhibited robust and monotonic convergence. By utilizing its complete spatial awareness of the 10×10 board, the agent consistently achieved an average reward of approximately 0.5 (Figure 5) and mantained a peak consumption rate exceeding 14 fruits per 100 iterations (Figure 6). This consistent intake allowed the fully observable agent to reach a stable average snake length plateau of approximately 18 to 19 segments (Figure 7), highlighting the effectiveness of the actor-critic policy when unconstrained by visual limitations.

On the other hand, restricting the agent's visual radius to a 5×5 ego-centric view significantly impaired its long-term pathfinding capabilities, resulting in highly noisy behaviour rather than a stable plateau. The partially observable A2C

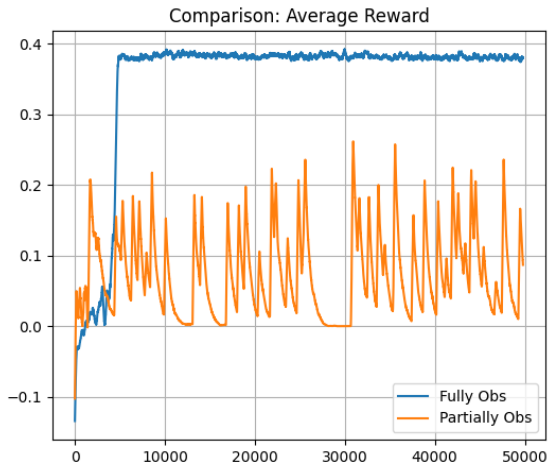

Figure 5. A2C Average Reward over 80,000 iterations. The fully observable agent displays a smooth convergence curve, while the partially observable agent is characterized by lower peaks and extreme variance.

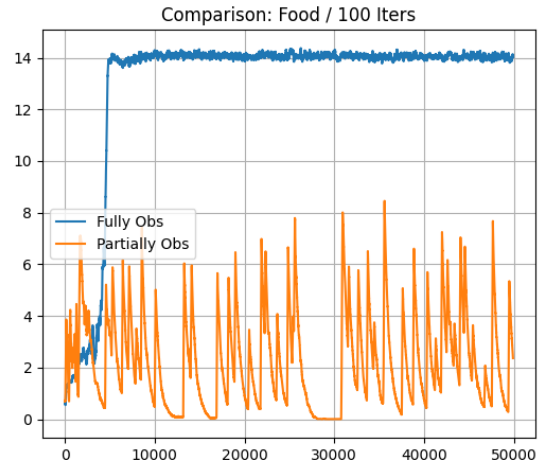

Figure 6. A2C Food eaten per 100 iterations. Global vision allows consistent intake, whereas partial observability forces an early, unstable plateau.

agent's performance metrics are extremely volatile; its average reward violently swings and frequently plunges toward 0 (Figure 5), while its food intake repeatedly drops near zero (Figure 6). Consequently, the agent's average length bounces wildly between 13 and 16 segments, never achieving true stabilization (Figure 7). This extreme variance is driven by a conservative, emergent survival strategy: once the snake's body size exceeds 14/15, it frequently traps itself in tight, repetitive self-loops in order to avoid fatal collisions in the unseen portions of the board. While this looping behaviour temporarily prolongs survival, it halts systematic growth and food consumption entirely, which explains the massive, noisy crashes consistently observed across all metrics.

6.1. Comparison: A2C Stochastic Policy vs. DQN Epsilon-Greedy

Although both the A2C and Double DQN architectures were ultimately limited by partial observability, a critical comparison of their exploration mechanics reveals their fundamentally different behavioural dynamics.

The DQN model relied on an ϵ-greedy strategy, which, in the partially observable setting, mandated a hard-coded minimum exploration rate ($\epsilon_{\min} = 0.1$). In contrast, the A2C model completely discards the ϵ-greedy heuristic in favor of stochastic sampling derived directly from its learned policy distribution, continuously regulated by a 0.01 entropy bonus. The high volatility observed in the A2C agent's average length and reward curves is not the result of forced

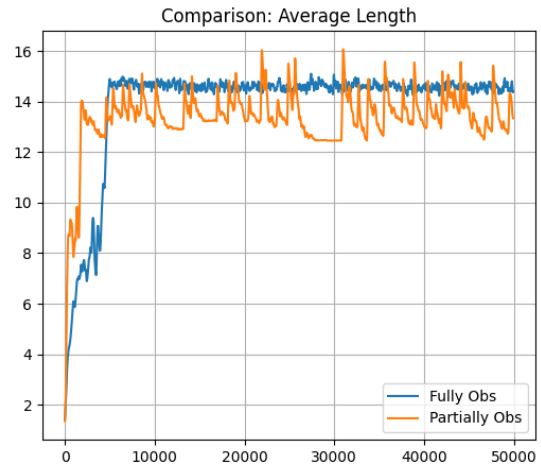

Figure 7. A2C Average Snake Length. The partially observable agent struggles to exceed 16 segments consistently, suffering frequent length drops due to self-collisions outside its field of view.

random number generation, but rather a reflection of genuine policy uncertainty. When the A2C agent's vision becomes obstructed by its own growing body, the actor network's confidence drops, naturally flattening the probabilities across the safe, masked logits. This allows the A2C agent to organically attempt cautious, probability-weighted navigation based on its generalized understanding of the environment, rather than executing arbitrary forced moves.

Table 1. Performance metrics summary comparing the Double DQN and A2C architectures across both environments, after training.

ENVIRONMENT	METRIC	DOUBLE DQN	A2C
FULLY OBS	AVG REWARD	$\sim$0.50	$\sim$0.38
FULLY OBS	FOOD / 100 ITERS	$\sim$14.5	$\sim$14
FULLY OBS	LENGTH RANGE	17.5 – 18.5	14.5 – 15
PARTIAL OBS	AVG REWARD	0.15 – 0.30	0 – 0.2
PARTIAL OBS	FOOD / 100 ITERS	6.0 – 9	0.5 – 8
PARTIAL OBS	LENGTH RANGE	12.5 – 17.5	12.5 – 15.5

7. Conclusion

In conclusion, the Double DQN utilizing a straightforward ϵ-greedy exploration strategy emerges as the practically superior approach. While both the DQN and A2C architectures achieve comparable performance ceilings, the DQN is fundamentally simpler to implement. It relies on a single network to predict expected rewards (Q-values) and optimizes a single, straightforward loss function.

In contrast, the A2C architecture introduces significant developmental complexity: it requires a dual-headed network to simultaneously act as a "player" (the actor predicting move probabilities) and a "judge" (the critic estimating state values). Furthermore, training this model requires a delicate, multi-part loss function that must carefully balance policy gradient updates, value predictions, and entropy bonuses. By avoiding this architectural overhead, the DQN provides a more robust and efficient framework for this specific task.

In conclusion, the performance plateau observed consistently across both models highlights the escalating difficulty of the Snake environment. As the agent succeeds, its growing body increasingly clutters the board. This drastically reduces the navigable state space and severely punishes spatial miscalculations. This dynamic scaling creates a difficult spatial challenge, proving that surviving the game's later, highly cluttered stages, remains an extremely difficult problem, even for advanced neural network architectures.